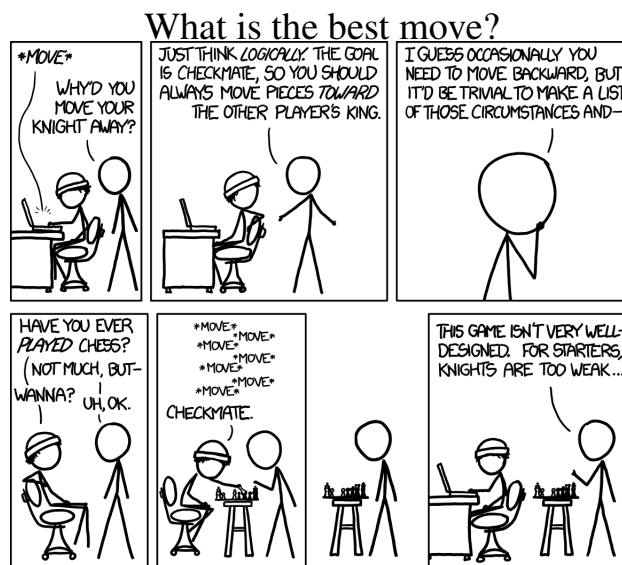


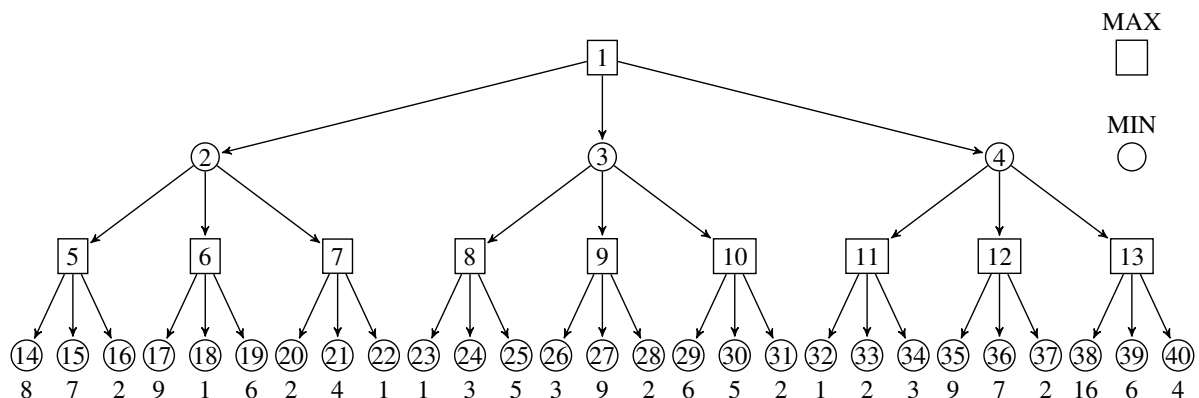
Tutorial No. 3

Adversarial Search



PART 1: Minimax search

Consider the following game tree. Player MAX plays first and is represented with rectangles; MIN player is represented with circles. Numbers in each node are names used for convenience to refer to them (starting node is node 1). Finally, utility of leaf nodes are shown below them (e.g., the utility of node 21 is 4).



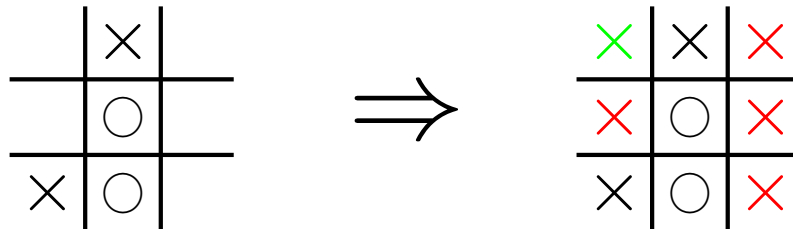
- Use minimax to determine the best move for MAX.
- Which nodes will not be examined if the alpha-beta procedure is used?

- (c) In which order will the nodes be examined by the alpha-beta procedure (assuming its depth-first implementation)?
- (d) Did the alpha-beta procedure give the same best move as minimax?
- (e) In general, does alpha-beta pruning change the result of mini-max? Prove your answer.
- (f) What are the properties of a “good” evaluation functions when using h-Minimax to avoid exploring the whole game tree?
- (g) Are there any analogues to admissibility/consistency for heuristics that result in a provably optimal search? Why/why not?
- (h) Is minimax search guaranteed to find the best move against any possible opponent? If not, could it be that minimax on some opponent give worse outcome than minimax on a fully rational agent?

PART 2: Expectimax search

Minimax search relies on the assumption that both players (MIN and MAX) will play the game optimally. If this were the case then every game of noughts-and-crosses would result in a draw — however, anyone who has played against a small child knows that this is not the case (I hope you let them win!).

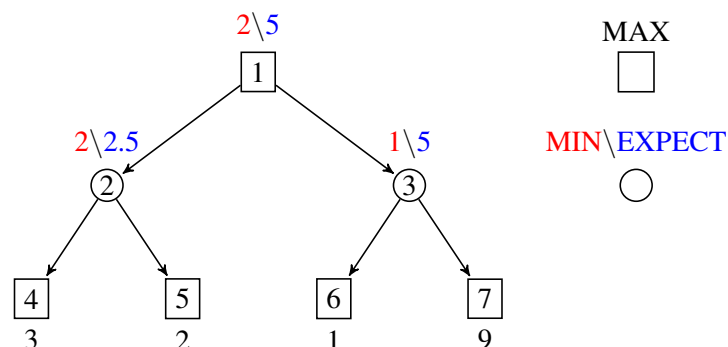
Take the following example of a game of noughts-and-crosses:



Here we have a situation on the left, where it is $\times$'s turn. If they put their cross in the top-left corner, then they are guaranteed to win in the next turn, however putting a cross in any other cell on the board is almost certainly going to result in a draw (provided $\bigcirc$ does nothing stupid). This is a reasonably trivial example, as anyone who has ever played the game before, can see that that this is the best move to make, but if it was the players first time playing — or the game was switched to something less trivial, like chess — the player might not see this as the optimal move, and put their cross somewhere else, allowing $\bigcirc$ to escape certain death.

It is in accounting for this uncertainty of play that is behind the idea of *expectimax* search. If minimax search is a model of the *worst-case* scenario for a given game, expectimax search is a model for the *average-case*. Here, instead of having MAX nodes and MIN nodes in the tree, we have MAX nodes and EXPECT nodes; with the EXPECT nodes taking the mean of the value of all of its children.

Consider the following example with minimax values in **red** and expectimax values in **blue**.



Here we can see that if minimax search is used then the left action will be chosen, with a utility of 2 and if expectimax is used, then the right action will be chosen with a utility of 5. Although it is possible that the opponent will choose the action that results in MAX obtaining a value of 1, the prospect of the possible 9 is enticing enough that it is worth the risk. It is important to note that there is no action that can be chosen that would ultimately result in a value of 5, this utility value is simply an average of the values that could be obtained if this action were chosen. The assumption here is that the opponent plays randomly; however, the actions can also be weighted to account for the fact that some choices are more likely than others. In the above example, when faced with two choices, one which will result in a utility of 1 for MAX and one which will result in a utility of 9 for MAX, even if the opponent is a very poor player, they will likely be able to distinguish between moves with such large differences in utility — and weighting these actions can account for this.

Another variant of this idea is *expectiminimax* search. Here, we have three types of nodes instead of two, MAX, MIN and EXPECT. Expectiminimax is used to model stochastic 2-player games, such as those where each player rolls a die and then makes a decision — the classic example used in the literature is backgammon. We will not go into detail about this technique, but it is mentioned here for the sake of differentiation.

- (a) Use expectimax search to determine the best move in the minimax game example from Part 1.
- (b) Is the result the same as you obtained in Part 1?
- (c) Can we use alpha-beta pruning with expectimax search?

End of tutorial worksheet